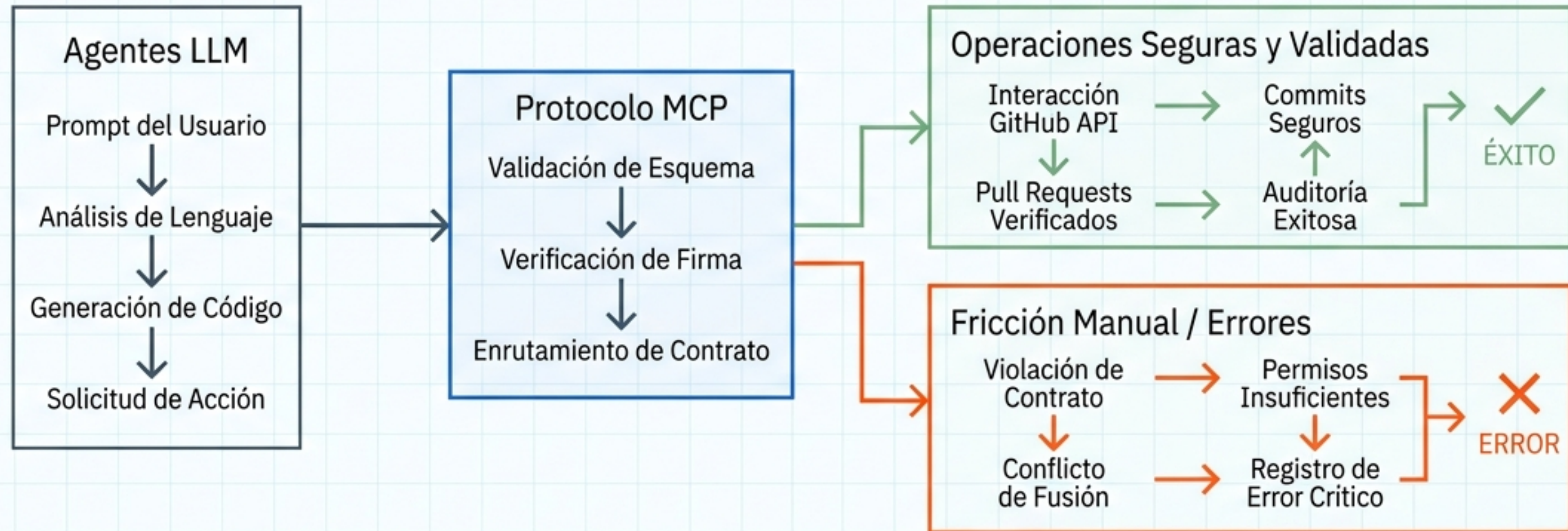


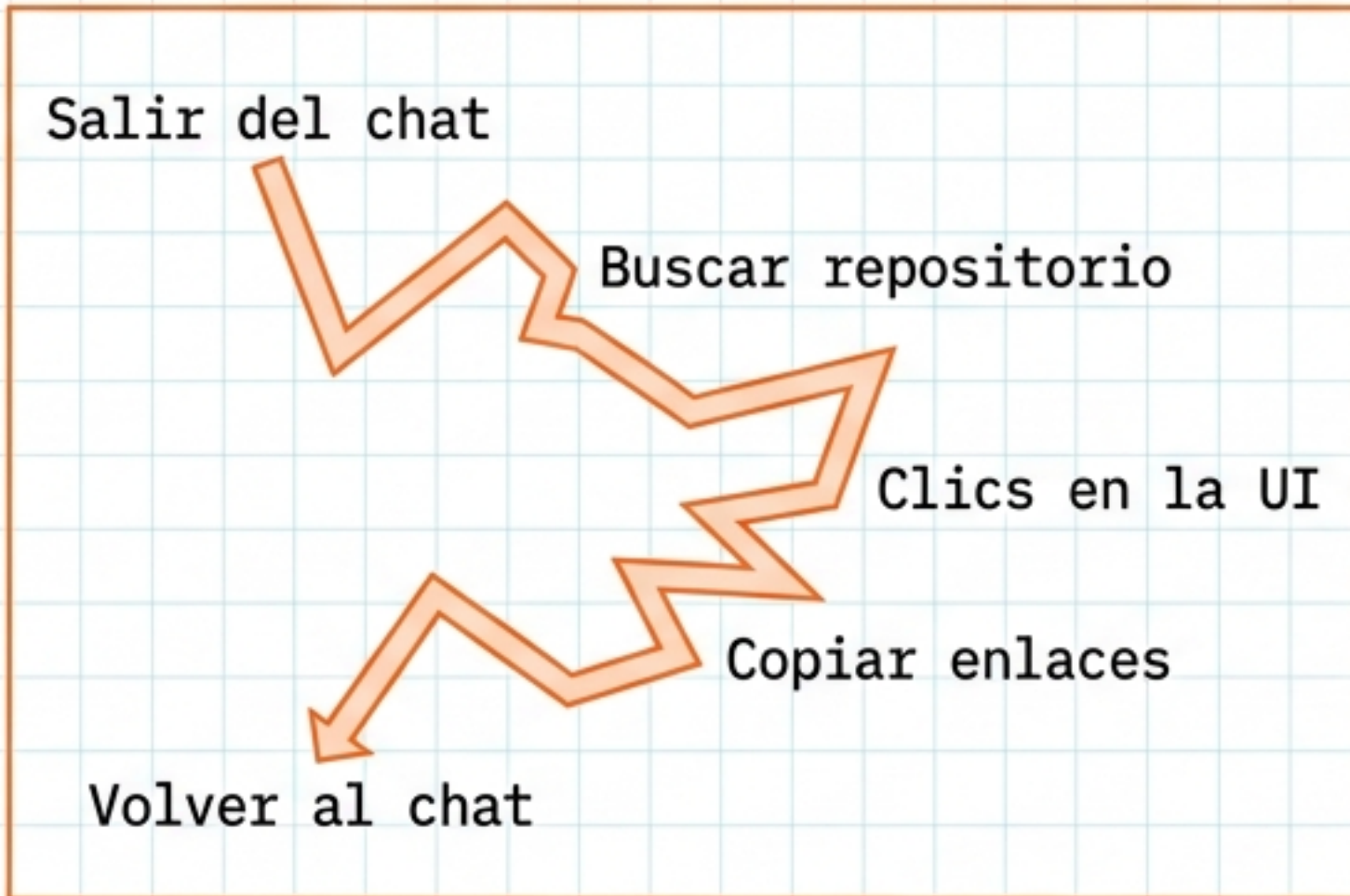
Automatización de GitHub vía MCP >_

Conectando agentes LLM con operaciones seguras y validadas mediante contratos estrictos.

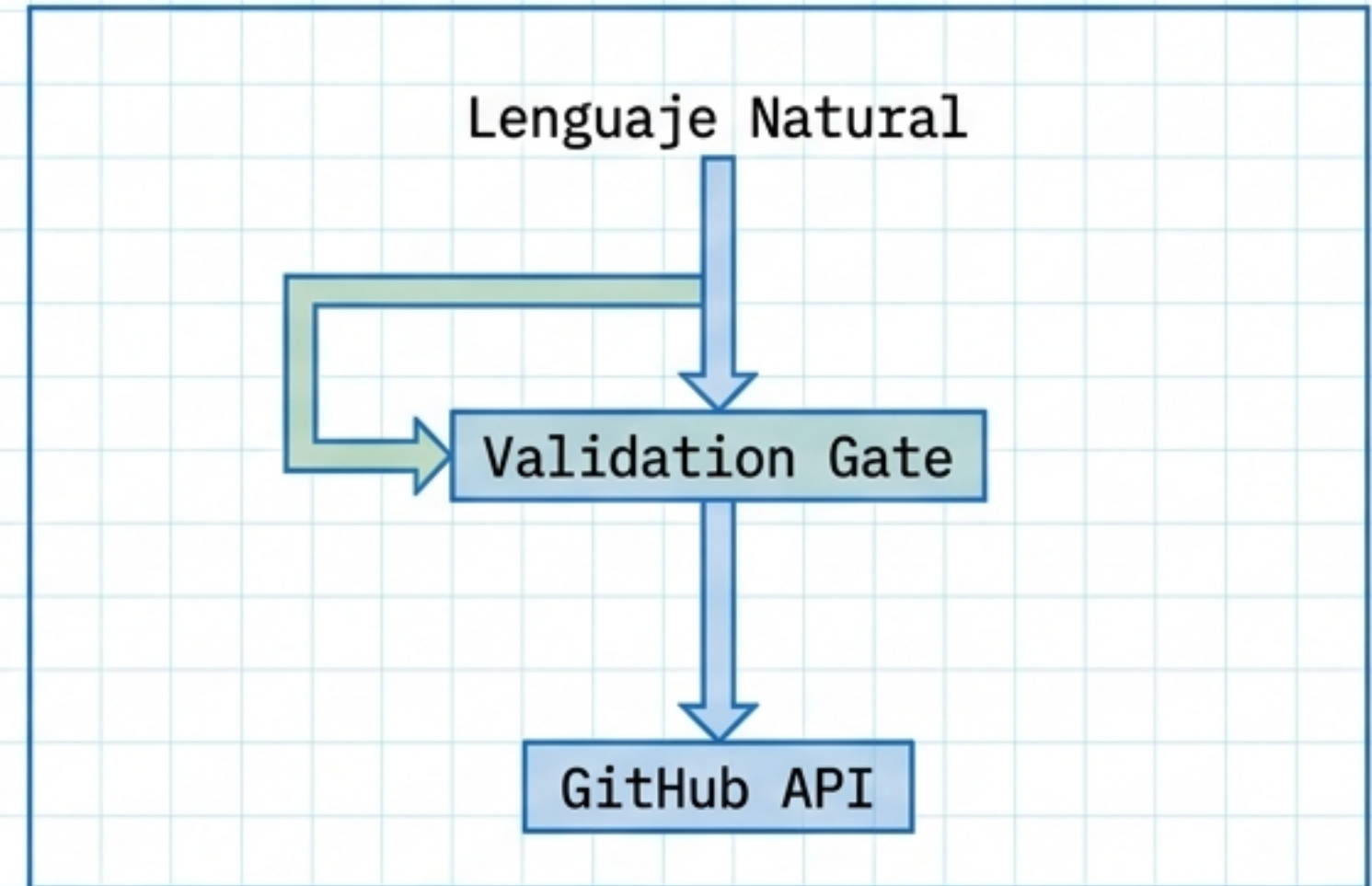


Lenguaje natural como interfaz, contratos estrictos como motor.

De la fricción manual a la ejecución unificada



El Problema: Las operaciones repetitivas rompen el estado de flujo de la conversación con el agente.



El Objetivo: Exponer herramientas MCP para que el LLM ejecute acciones sin abandonar el chat.

El LLM decide qué operación necesita, pero el servidor conserva el control absoluto sobre qué datos acepta y cómo se ejecutan.

El catálogo de herramientas expuestas al LLM



``create_repository``

Crea un repositorio privado o público para el usuario autenticado.



``create_issue``

Abre un issue con título, descripción y etiquetas específicas.



``list_repositories``

Consulta y enumera los repositorios del usuario.



``create_commit``

Crea un archivo nuevo o actualiza uno existente mediante un commit directo.

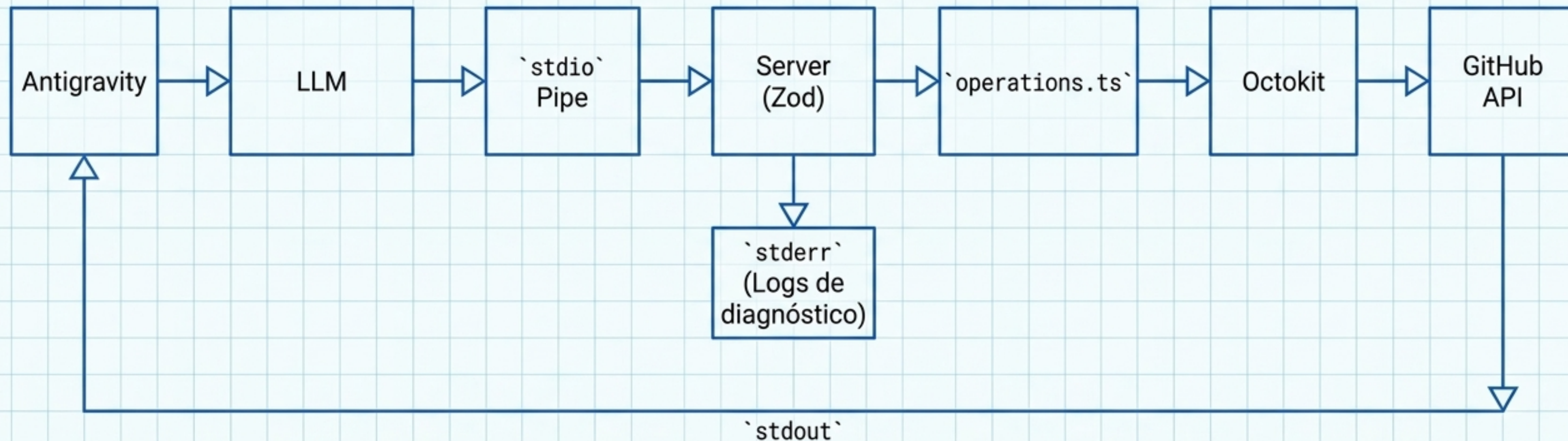


``list_issues``

Filtra y lista issues por estado dentro de un repositorio.

Todas las herramientas operan bajo la autoridad del usuario autenticado y requieren payloads estructurados.

Anatomía y flujo de una petición MCP



1. Host envía petición MCP por `stdio`.

2. El LLM interpreta la intención natural.

3. El SDK entrega los argumentos en JSON.

4. Zod aplica validación estricta de frontera.

5. La lógica de negocio decide la acción.

6. Octokit maneja la red y los reintentos (429/5xx).

7. GitHub API ejecuta y responde.

8. El resultado vuelve como texto enriquecido al agente.

Alineación entre tecnología y necesidad estructural

Tecnología	Propósito Arquitectónico	Justificación
TypeScript	Contratos explícitos	Reduce errores de tipado entre schemas, herramientas y operaciones finales.
Protocolo stdio	Aislamiento de red	Servidor local lanzado por el host sin necesidad de administrar puertos HTTP abiertos.
Zod	Validación de frontera	Bloquea nombres inválidos, rutas inseguras y tamaños excedidos antes de tocar la lógica.
Octokit	Encapsulación	Maneja la autenticación y los endpoints REST sin requerir construcción manual de peticiones.

Disecccionando el registro de una herramienta

```
server.registerTool("list_issues", {  
  description: "Lista los issues de un repositorio...",  
  inputSchema: schemas.listIssuesSchema.shape,  
}, (input) => run(async () =>  
  (await operations.listIssues(client, input)).data  
));
```

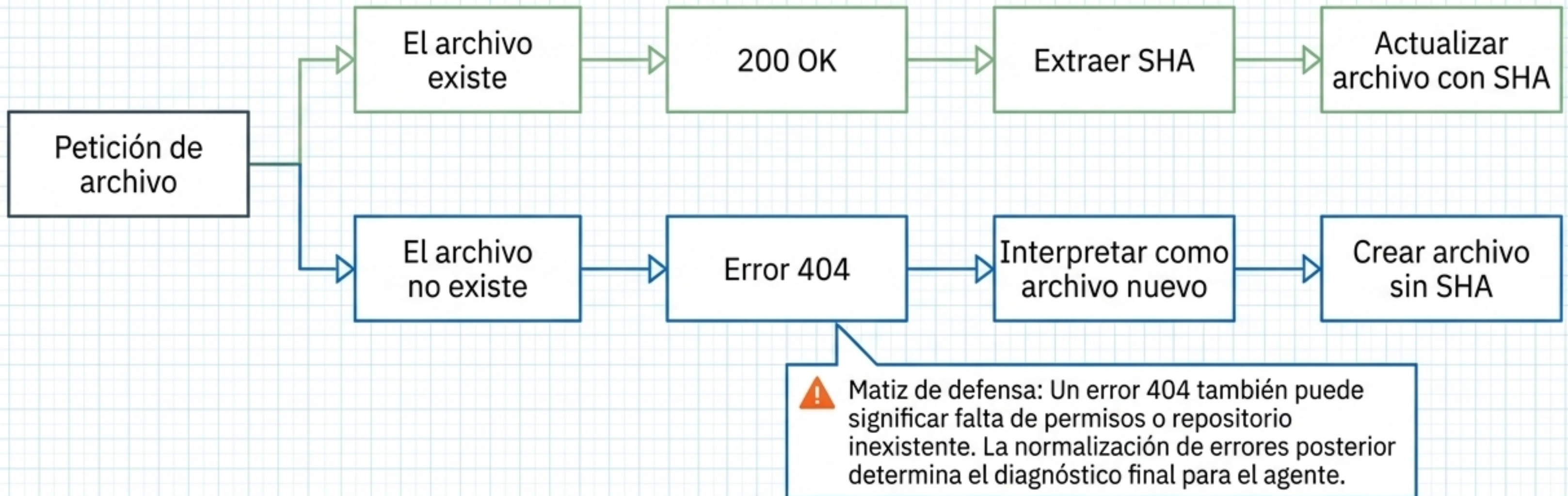
El prompt del sistema: La descripción guía al LLM para que sepa exactamente cuándo y cómo usar la herramienta.

El escudo protector: Zod limita y purifica los argumentos; si el LLM alucina un formato, la petición se rechaza aquí.

El orquestador: Separa la operación de red del protocolo MCP. Centraliza el éxito, el registro de errores y la respuesta.

Resolución de bifurcaciones lógicas complejas

La operación `create_commit` debe adaptar su payload dinámicamente según el estado del repositorio remoto. GitHub exige el SHA del blob actual para actualizar, pero lo rechaza al crear.



Capas de resiliencia y seguridad del servidor



Autenticación Ciega

El GITHUB_TOKEN reside exclusivamente en el entorno local.

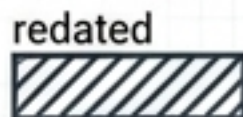
Jamás se recibe como argumento en un schema.



Observabilidad Aislada

Los logs fluyen hacia `stderr` para proteger el canal MCP en `stdout`.

Redacción automática de secretos y tokens.



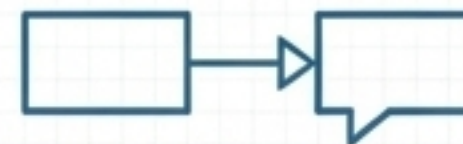
Tolerancia a Límites

Lógica de reintento ante errores 429 y 5xx. Respeto estricto por el header `retry-after`, con respaldo de backoff exponencial.

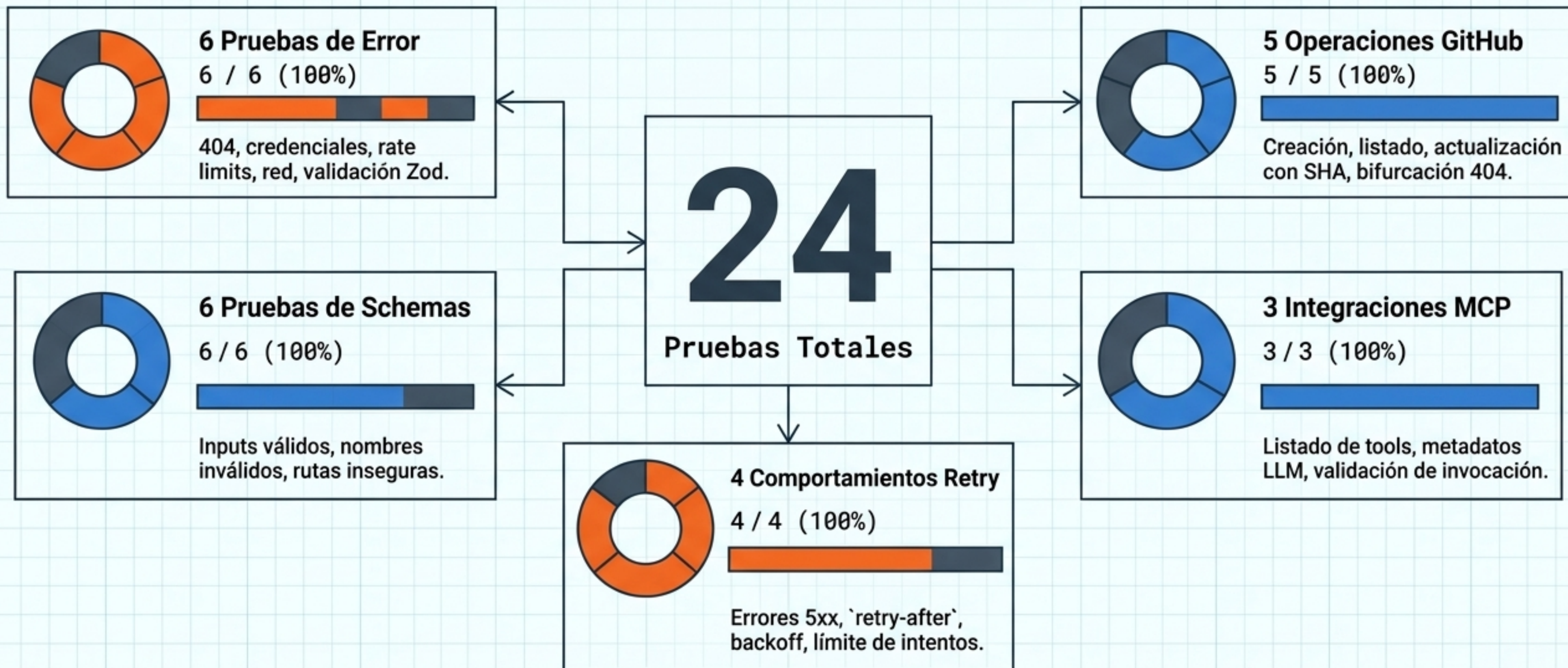


Mensajería Accionable

Conversión de errores técnicos de API en instrucciones claras para el LLM (ej: sugerir verificar permisos ante un 404).

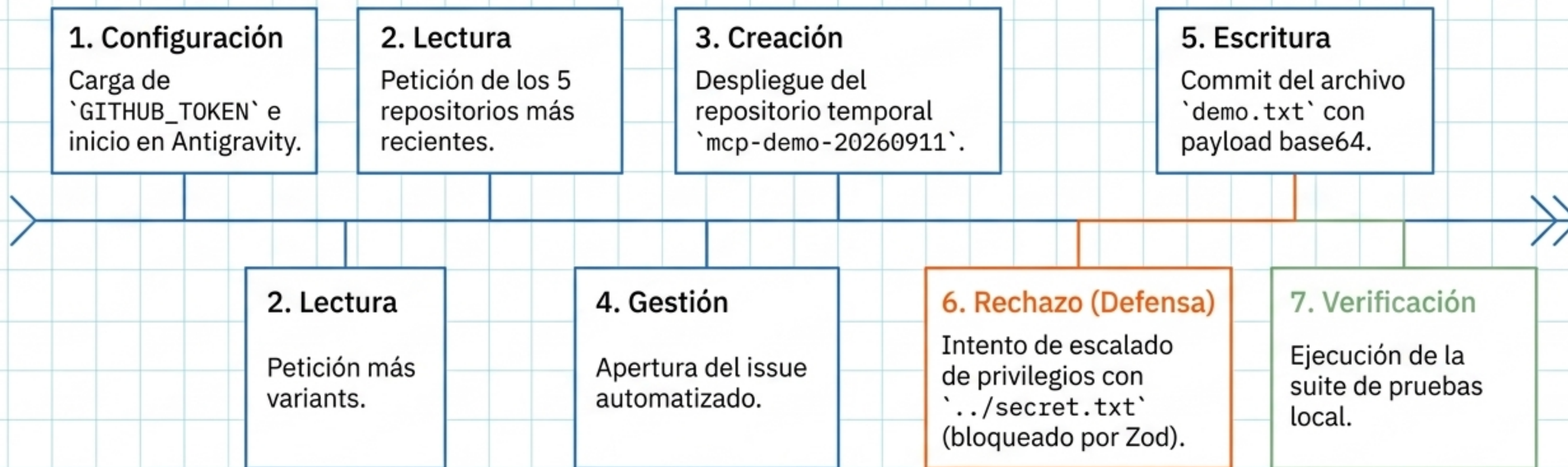


Cobertura determinista mediante inyección de dependencias



Mockeo de Octokit + MCP en memoria = Pruebas 100% deterministas sin efectos secundarios en repositorios reales.

Secuencia de la demostración en vivo



Fricciones arquitectónicas y su resolución

Fricción: GitHub requiere SHA para actualizar, pero no para crear.

Solución: Patrón de doble ruta; consulta previa con asimilación del error 404.

Fricción: Cualquier log contamina y rompe la comunicación stdio.

Solución: Aislamiento total; stdout exclusivo para MCP, diagnósticos a stderr.

Fricción: Los errores HTTP de Octokit confunden al agente LLM.

Solución: Centralización en toAppError para generar respuestas en lenguaje natural accionable.

Fricción: Probar contra la API real produce tests frágiles y lentos.

Solución: Inyección de dependencias para mockear completamente la capa de Octokit.

Defensa de decisiones de diseño

Q



A

¿Por qué `stdio` y no HTTP?

El servidor es un sidecar local del host MCP. `stdio` elimina la gestión de puertos y la sobrecarga de autenticación de red local.

Q



A

¿Cómo se protege el Token?

Se extrae directamente del entorno del proceso. No existe en ningún schema y el logger bloquea activamente cualquier cadena similar a un secreto.

Q



A

¿Cuál es la política de Rate Limit?

Distinción estricta: Errores 429 usan `retry-after` nativo o backoff exponencial. Errores 403 por permisos fallan inmediatamente sin reintento.

Q



A

¿Estrategia de escalabilidad futura?

Aislar nuevas operaciones en `operations.ts`, registrar el schema Zod y añadir tests unitarios de contrato, manteniendo la capa MCP intacta.

Principios para el desarrollo de servidores MCP

Contratos como escudos

La validación estricta de frontera evita que las alucinaciones del LLM se conviertan en errores destructivos en la API externa.

Separación de responsabilidades

El protocolo MCP permite que el agente conserve la agencia (decisión) mientras el servidor retiene la autoridad (ejecución y seguridad).

Errores orientados al consumidor

Un sistema diseñado para LLMs debe devolver errores que guíen al modelo hacia la corrección, no simples trazas de pila técnicas.

Convirtiendo la incertidumbre del lenguaje natural en operaciones controladas, predecibles y auditables.